

Performance Engineering

July 15th 2026

Preliminaries

Who Am I

Maximilian Kuschewski
Software Engineer at TigerBeetle

- Formerly PhD Student at TUM
- Focus on high performance query and transaction processing on modern cloud hardware
- Joined TigerBeetle in May 2026

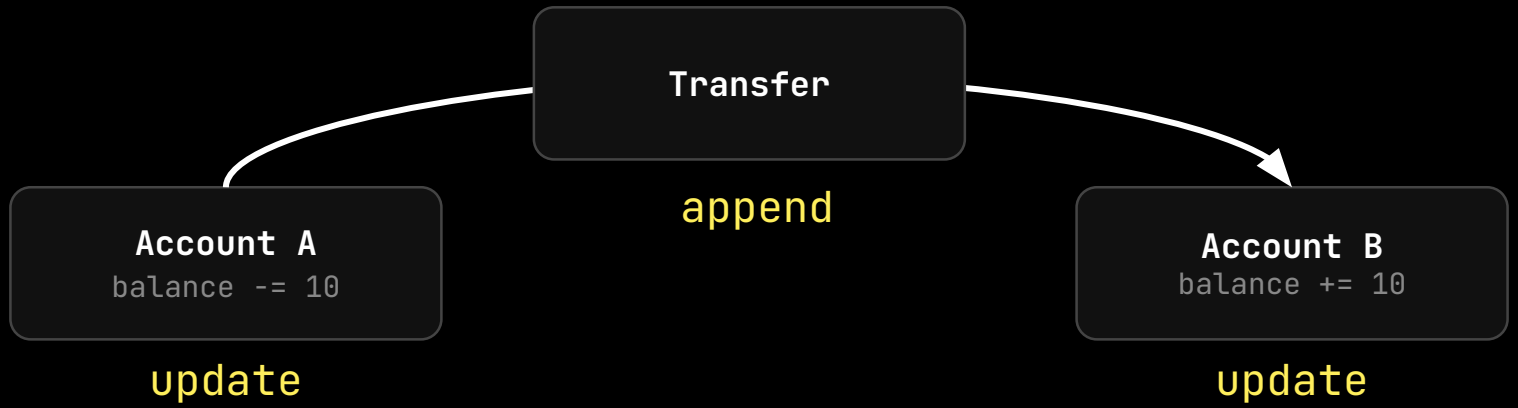
What is TigerBeetle

Financial transactions database built for safety and 1000x performance.

- Designed for double-entry accounting and high debit/credit throughput (OLTP)
- Strict Serializability
- RSM on top of an LSM
- Written in Zig

Background: Transactions

Inventory Counts, Shopping Cart Items,
Checkout Payments, Deliveries, Refunds,
Kilowatt Hours, Trades, Airplane Seats



Understand your Workload

Inventory Counts

Shopping Cart Items

Checkout Payments

Deliveries

Refunds

Kilowatt Hours

Trades

Airplane Seats

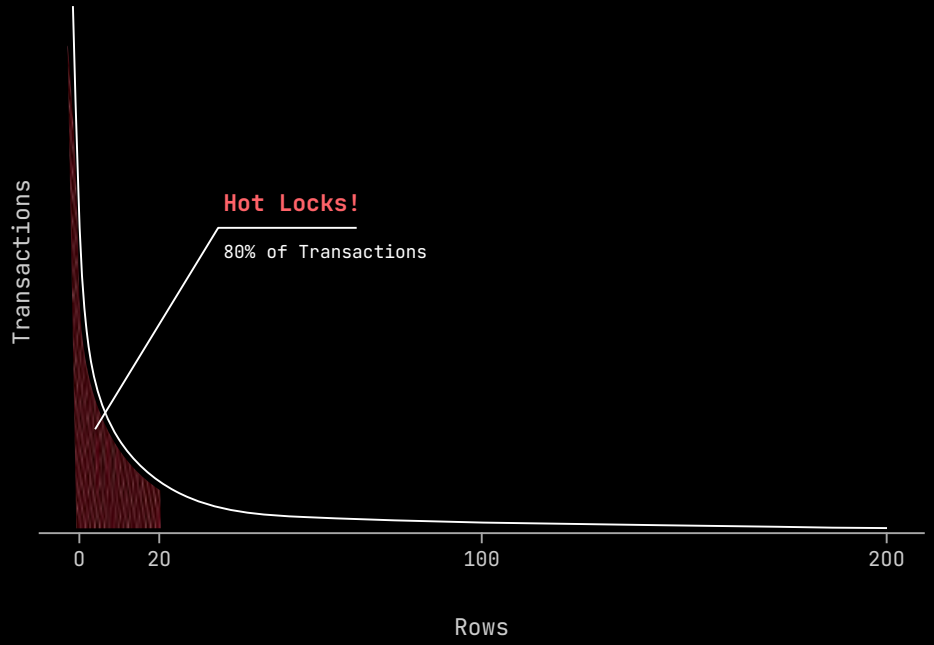
Many updates

Tiny payloads

High volume

High contention

Power Laws Everywhere



Transfers in the Wild



Performance

We'll change this today!

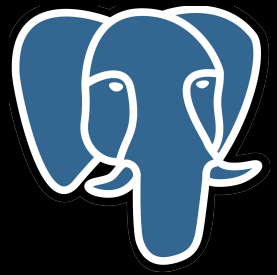


100ms

p100 latency

400,000 TPS

any contention



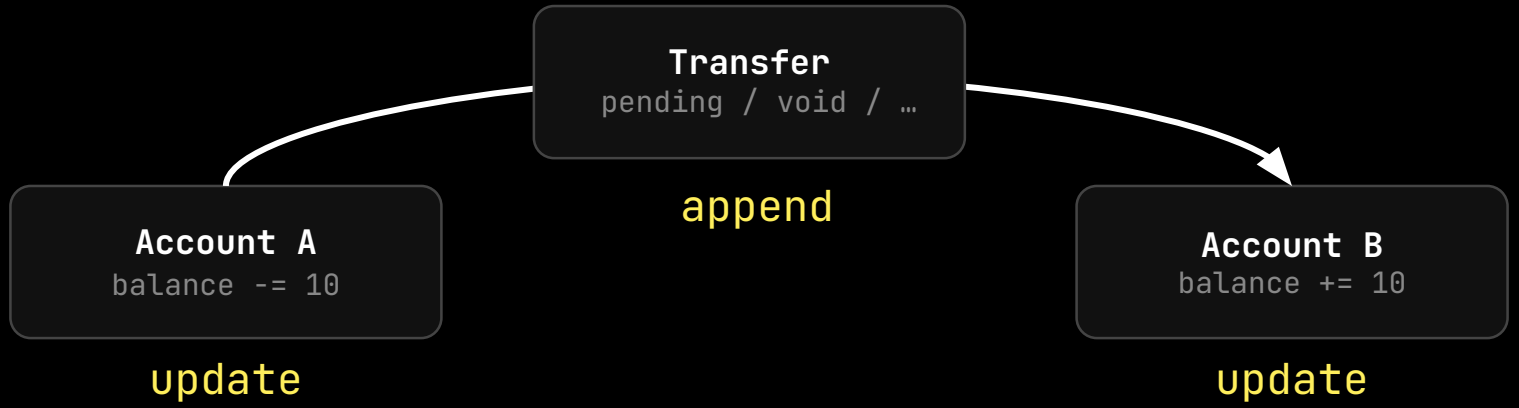
10,000 TPS

low contention

100 TPS

high contention

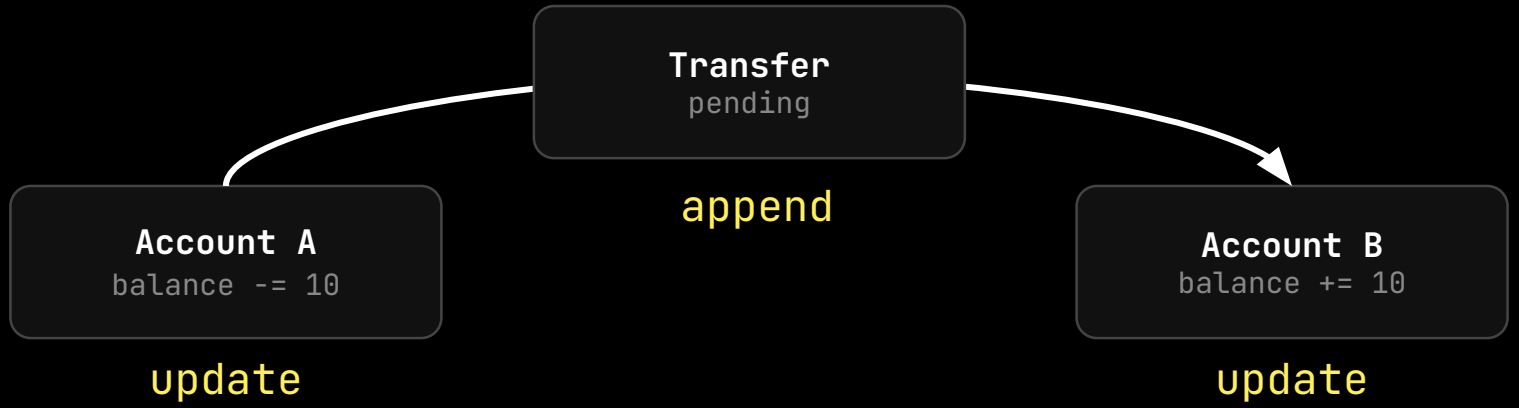
Static Interface




```
create_account ( account    )
lookup_account ( account_id )

create_transfer ( transfer    )
lookup_transfer ( transfer_id )
```

Interface



Transactions Inside Batches

```
create_accounts( accounts     )
lookup_accounts( account_ids  )

create_transfers( transfers    )
lookup_transfers( transfer_ids )
```

Autobatching Client

TIGER STYLE

Safety

Performance

Experience

in that order.

The best time to solve performance, **to get the huge 1000x wins**, is in the design phase.

Perform back-of-the-envelope sketches.

All memory must be statically allocated at startup. Put a limit on everything because, in reality, everything has a limit.

Don't react directly to external events. Your program should run at its own pace.

Zero dependencies, apart from the Zig toolchain.



TIGER STYLE

Walk through the building,
look around,
constantly improve things.



Tail Latency

P50	15 ms
P99	70 ms
P100	120 ms

1ms network latency

8189 transfers per batch

Is this good?

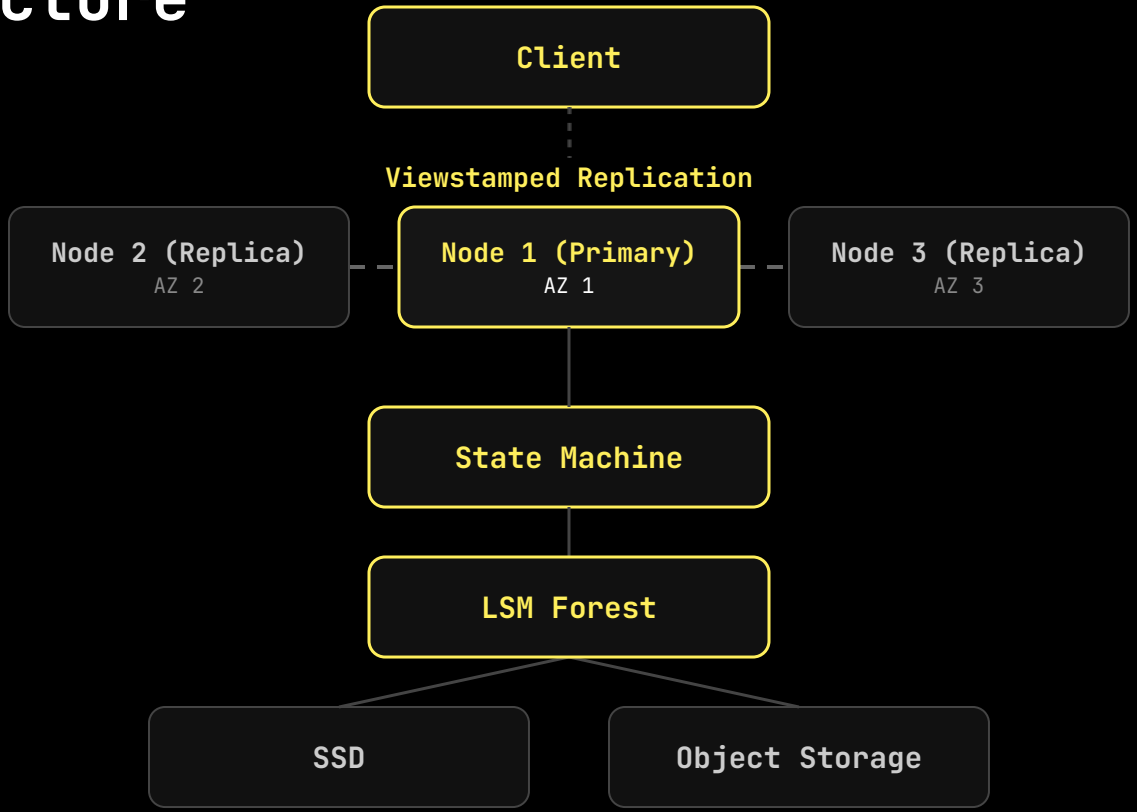


Back of the Envelope: Execution

1MB	SSD write (WAL)		~0.2ms
1RTT	for consensus		1ms
16k	SSD reads?	(cached in-memory)	-
16k	account lookups	(16k*2*128B/10GB/s)	< 1ms
8k	idempotency checks	(filter key range)	-
8k	transfer execution		< 1ms

It’s not replication or execution

Architecture



Latency Trace

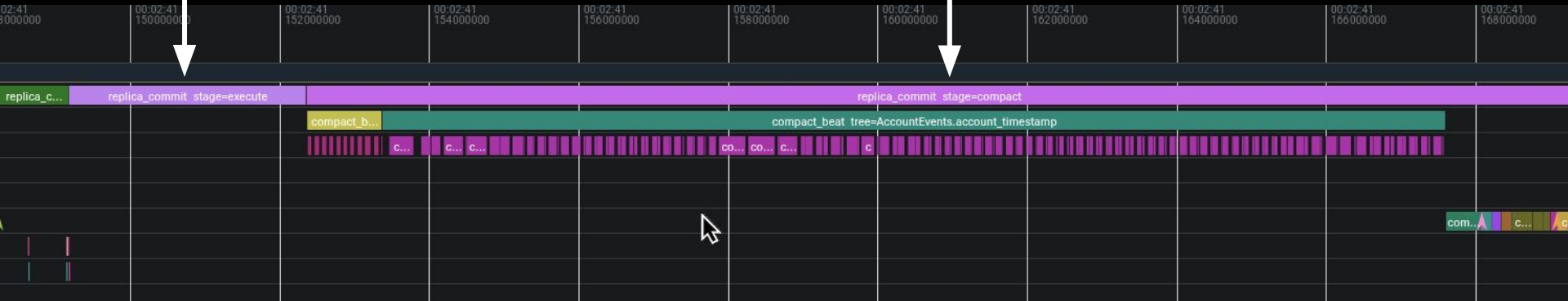
⛓🔗 ui.perfetto.dev

Execution

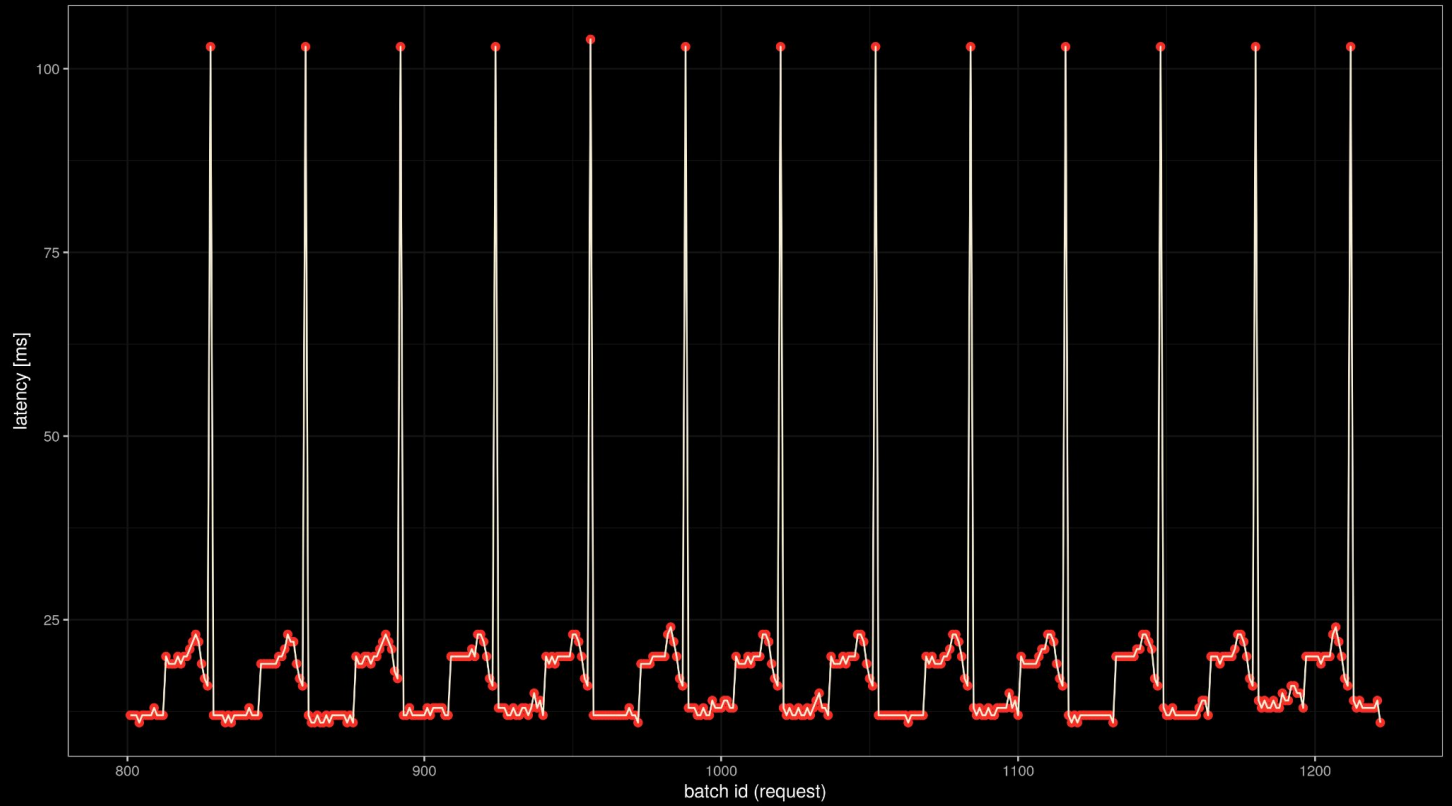
3.5ms

LSM Compaction

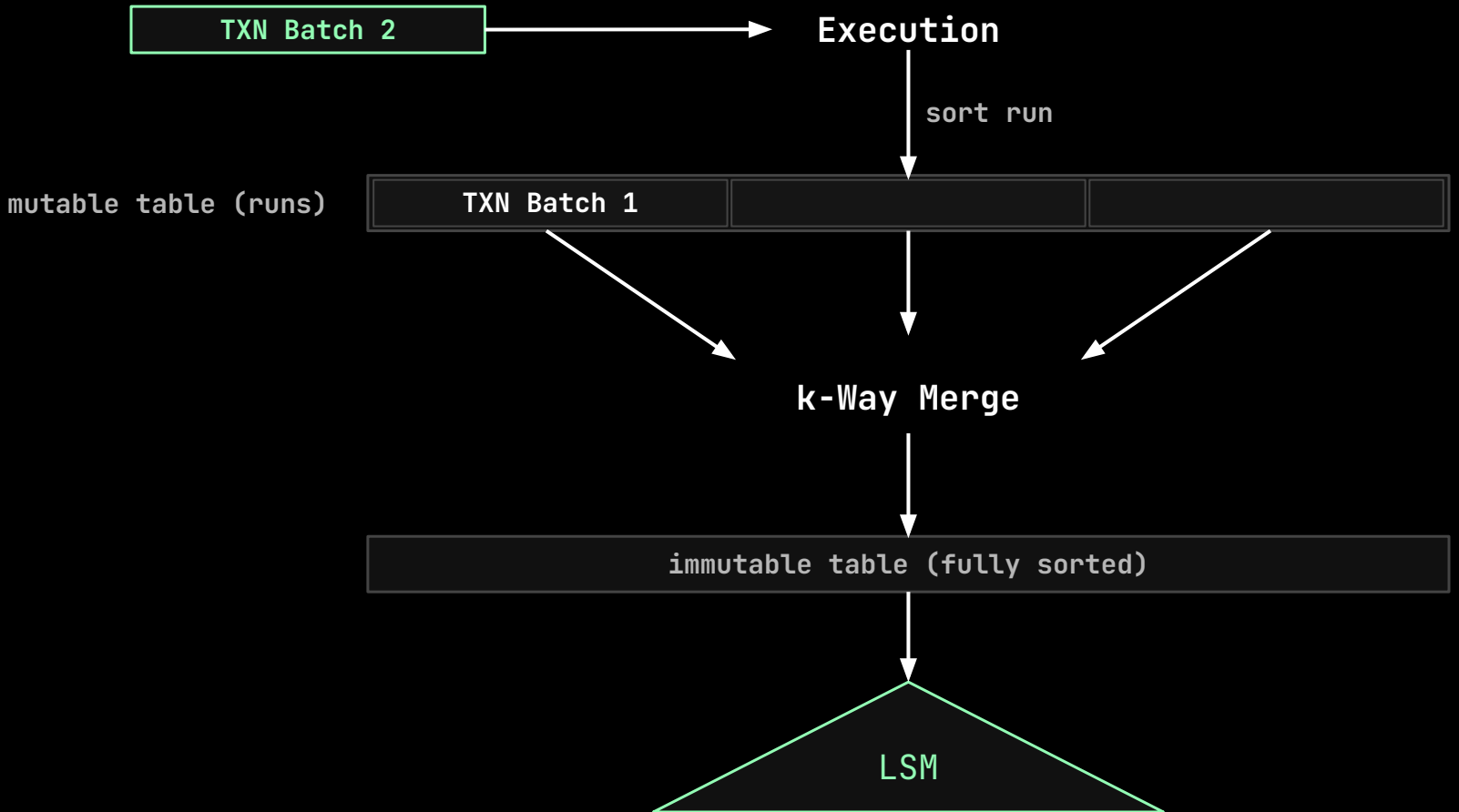
16.5ms



Tail Latency



Storing Batches



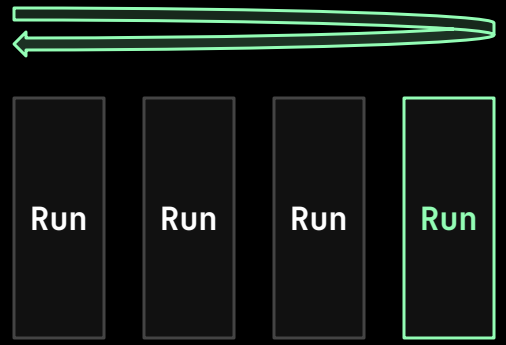
k-Way Merge

Merge **k sorted runs** into **one sorted** sequence.

Naïve approach:

- Scan all *k* runs to find the minimum element.
- Repeat until all *n* elements are merged.

Time complexity: **O(n · k)**



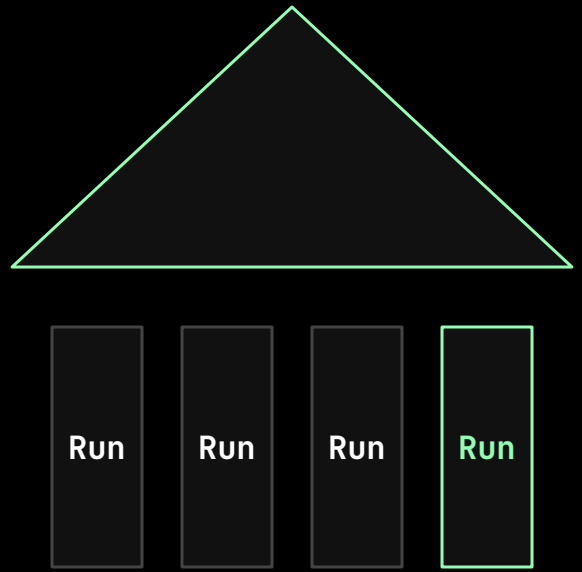
k-Way Merge

Merge **k sorted runs** into **one sorted** sequence.

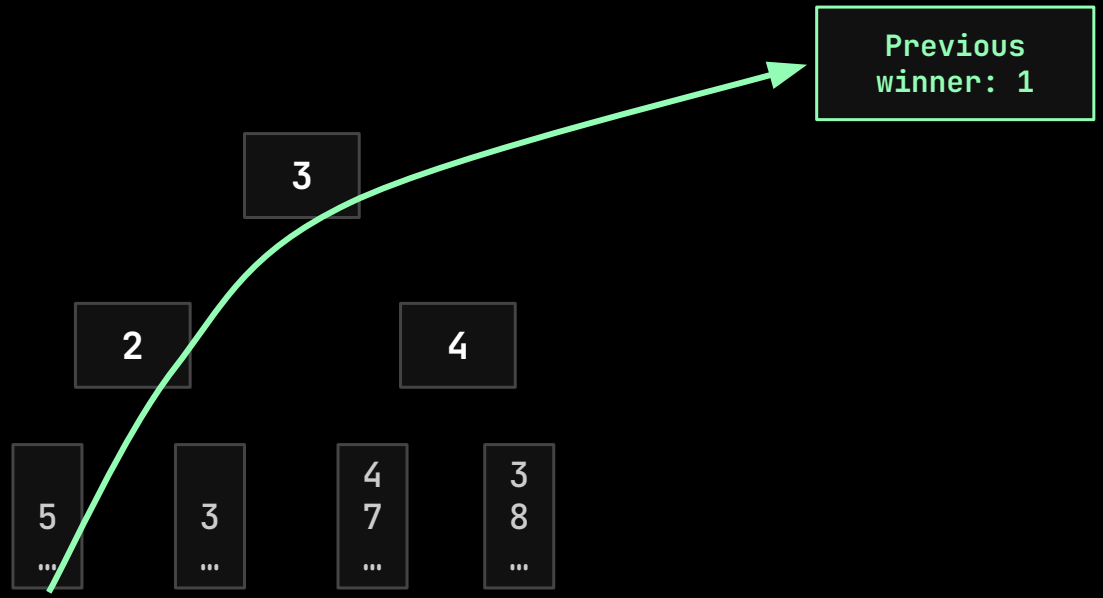
Tree-Based approaches:

- Select min-element in log(k).
- Repeat until all *n* elements are merged.

Time complexity: **O(n · log(k))**



Tree of Losers (Tournament Tree)



Only one path competes → **1 comparison** per level (vs. 2 in a min-heap). Asymptotically optimal.

Did we fix latency?

	before	after
P50	15ms	18ms
P99	70ms	80ms
P100	120ms	159ms

Algorithm Engineering

perf_event in the wild!

name	k	cycles	instr.	cache_ref	cache_miss	branch_miss	IPC
std.sort	32	240.5	324.9	24.8	0.42	2.75	1.35
loser tree	32	276.4	625.6	3.3	0.17	2.53	2.26

Napkin Math:

Tree height: log2(32) = 5; 50% mispredictions = 2.5 misses

Algorithm Engineering

name	k	cycles	instr.	cache_ref	cache_miss	branch_miss	IPC
std.sort	32	240.5	324.9	24.8	0.42	2.75	1.35
loser tree	32	276.4	625.6	3.3	0.17	2.53	2.26



./perf
record



replace
std.mem.swap



@branchHint(.unpredictable)



convince compiler to
generate branchless code

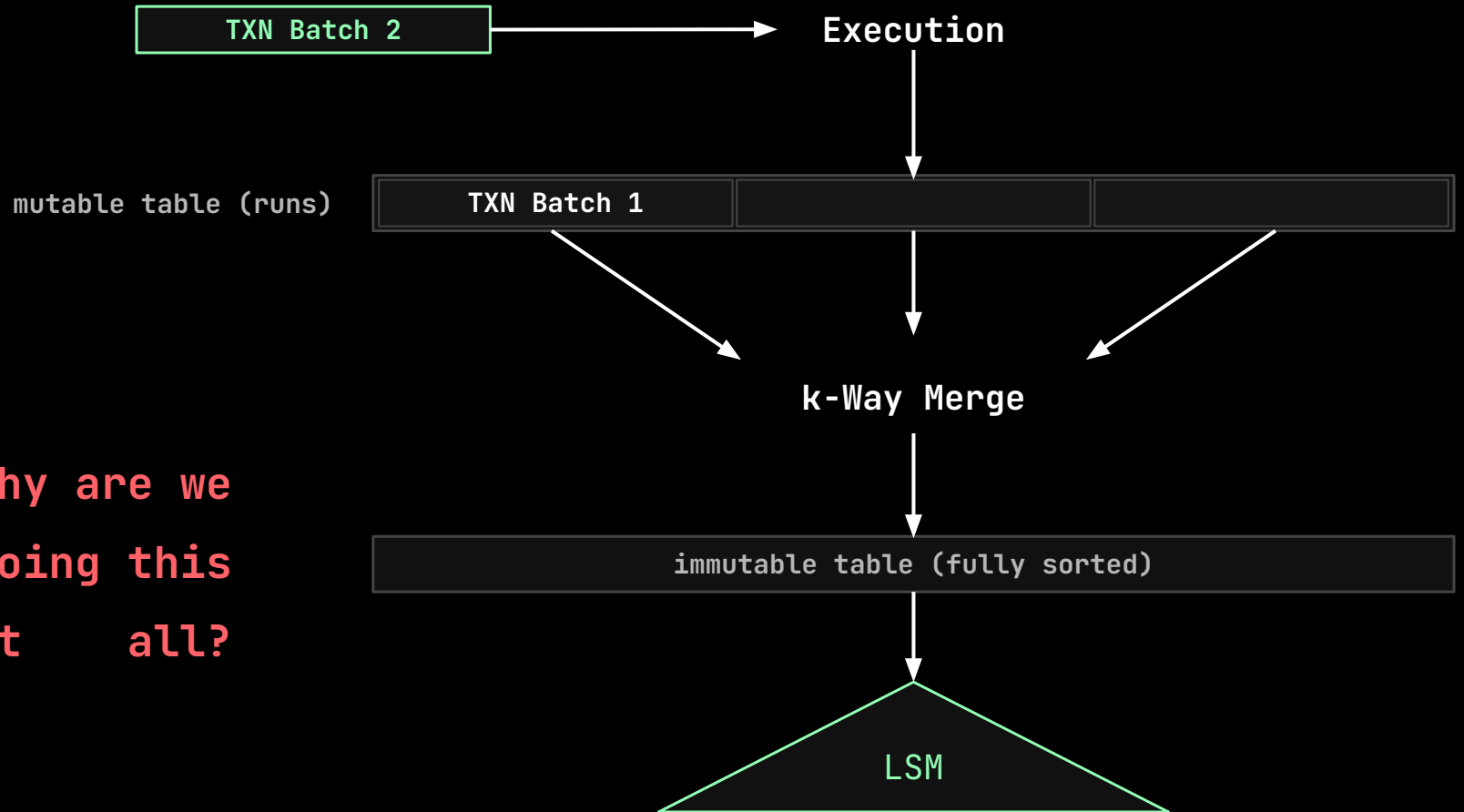
Algorithm Engineering

name	k	cycles	instr.	cache_ref	cache_miss	branch_miss	IPC
std.sort	32	240.5	324.9	24.8	0.42	2.75	1.35
tree v1	32	276.4	625.6	3.3	0.17	2.53	2.26
tree v2	32	84.1	391.8	2.9	0.11	0.02	4.66

Did we fix latency?

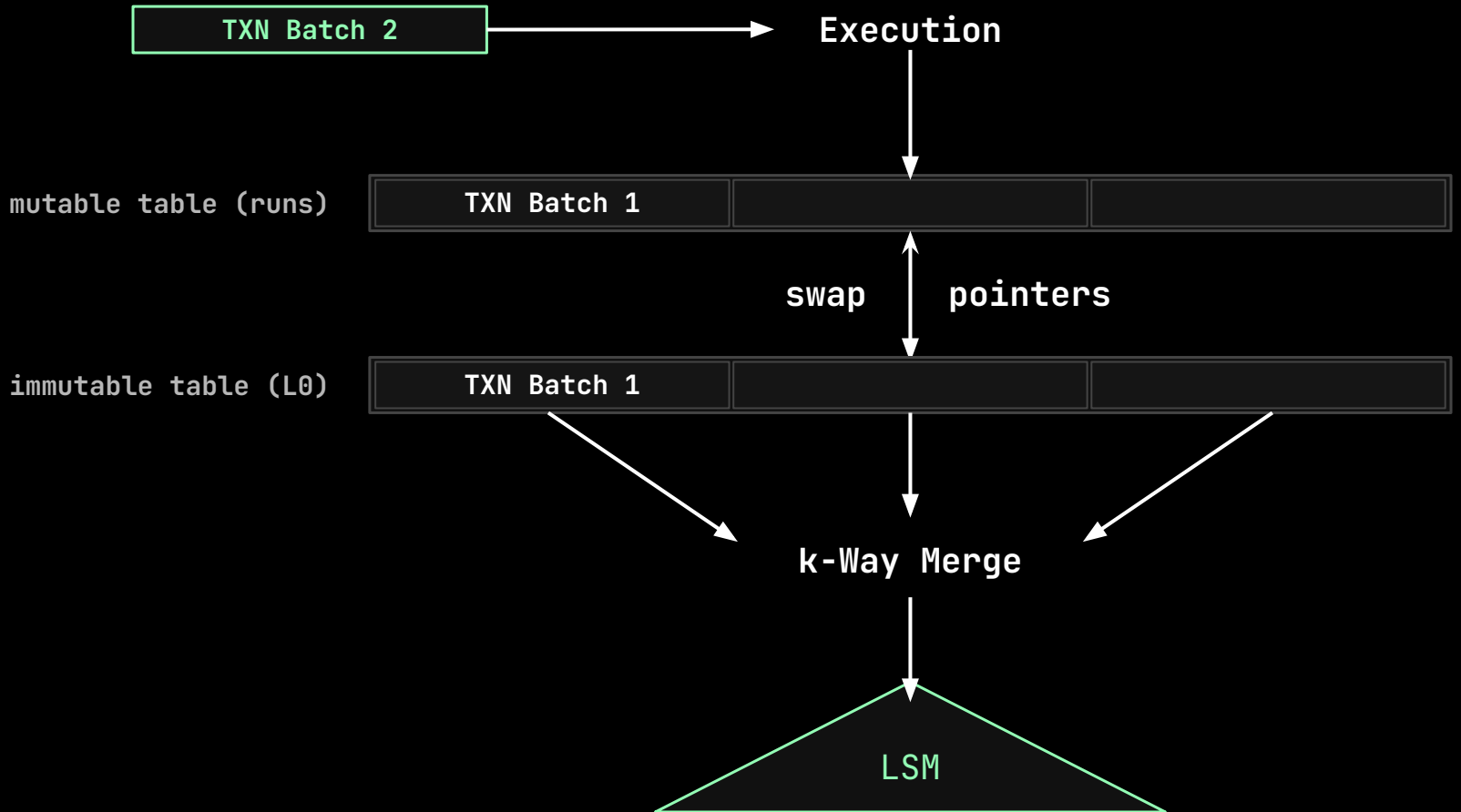
	before	tree
P50	15ms	15ms
P99	70ms	50ms
P100	120ms	70ms

Do things more efficiently...



Why are we doing this at all?

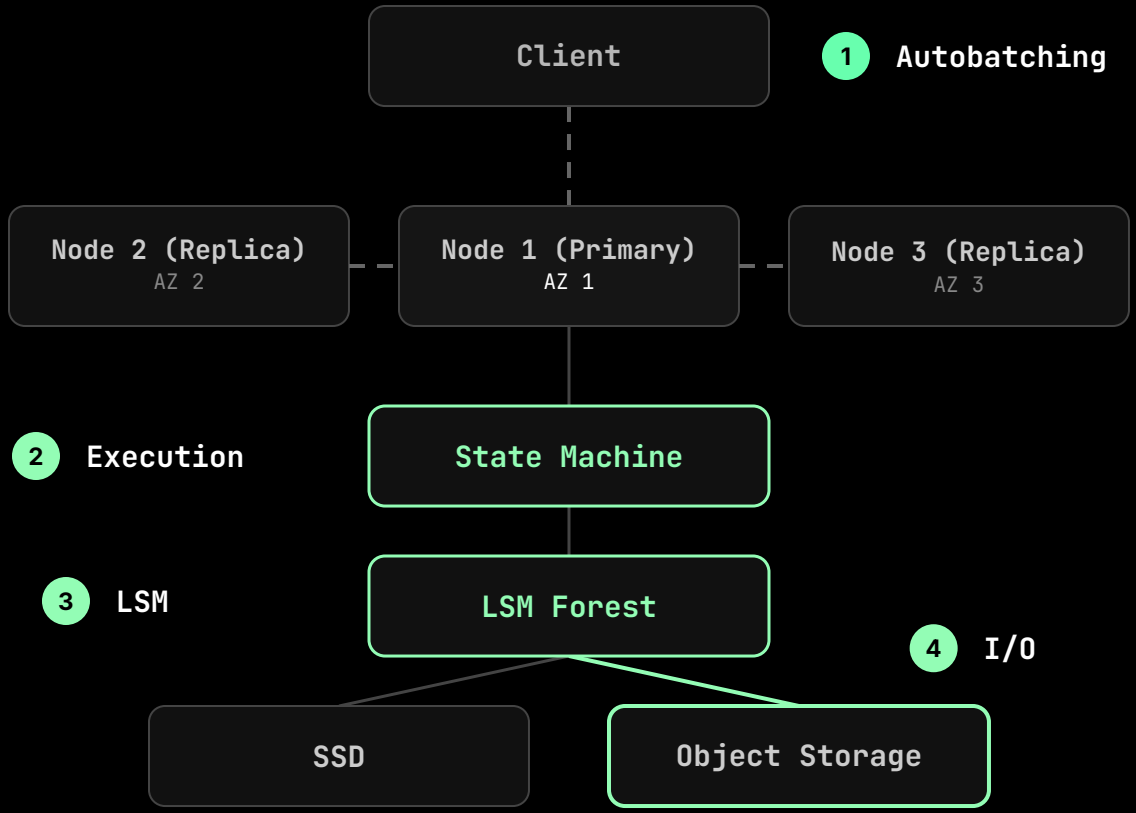
...or just do less!



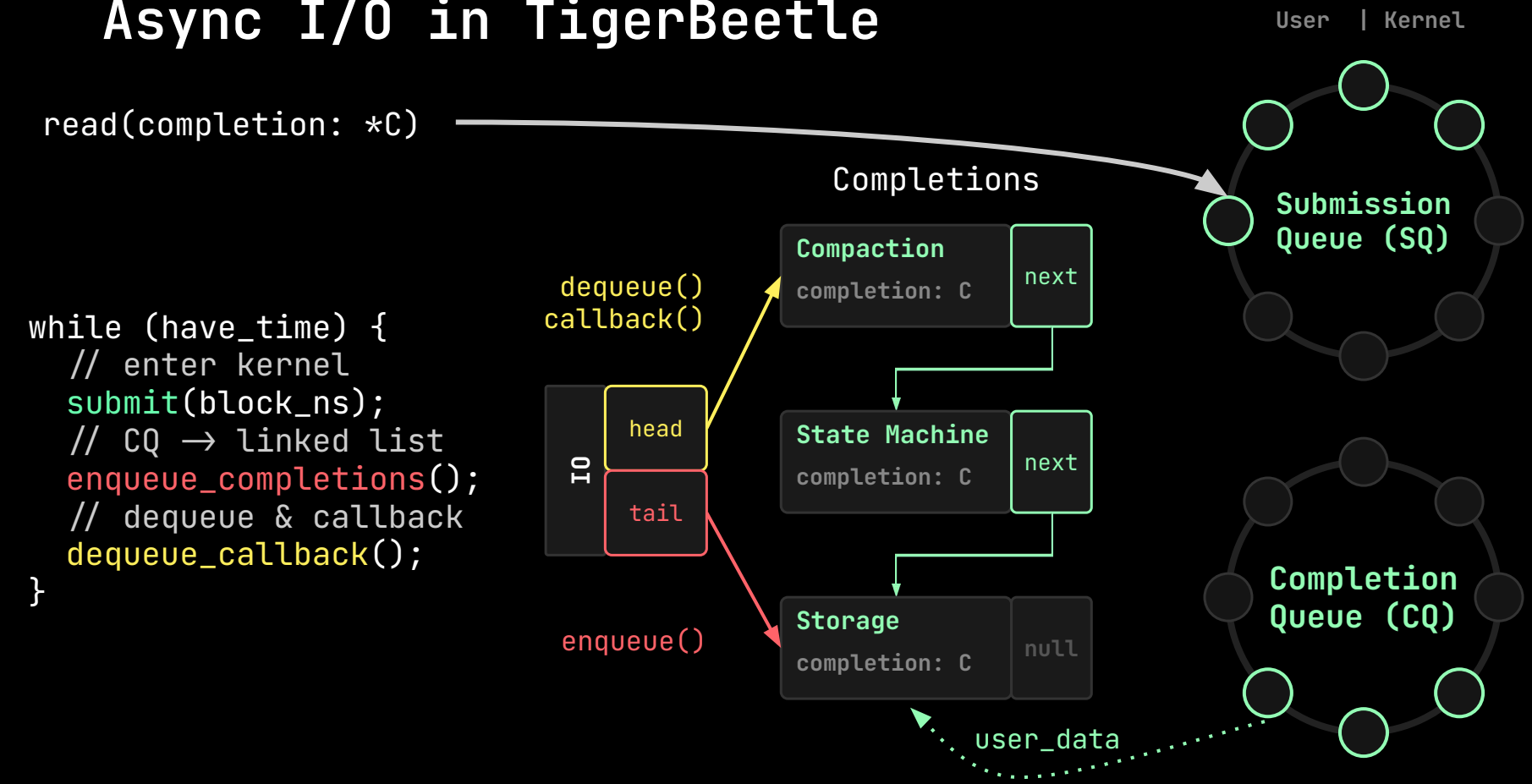
Did we fix latency?

	before	tree	lazy
P50	15ms	15ms	18ms
P99	70ms	35ms	30ms
P100	120ms	70ms	35ms

Up Next


Client
1 Autobatching
Node 2 (Replica)
AZ 2
Node 1 (Primary)
AZ 1
Node 3 (Replica)
AZ 3
2 Execution
State Machine
3 LSM
LSM Forest
4 I/O
SSD
Object Storage

Async I/O in TigerBeetle



Object Storage SDKs

- Are often careless with blocking syscalls
  - AWS: DNS lookups
- Use many other libraries internally
  - AWS: curl, encryption, …
- Spawn their own thread pools
- Allocate, deallocate, copy, …



Object Storage Clients



S3 / Azure / GCP

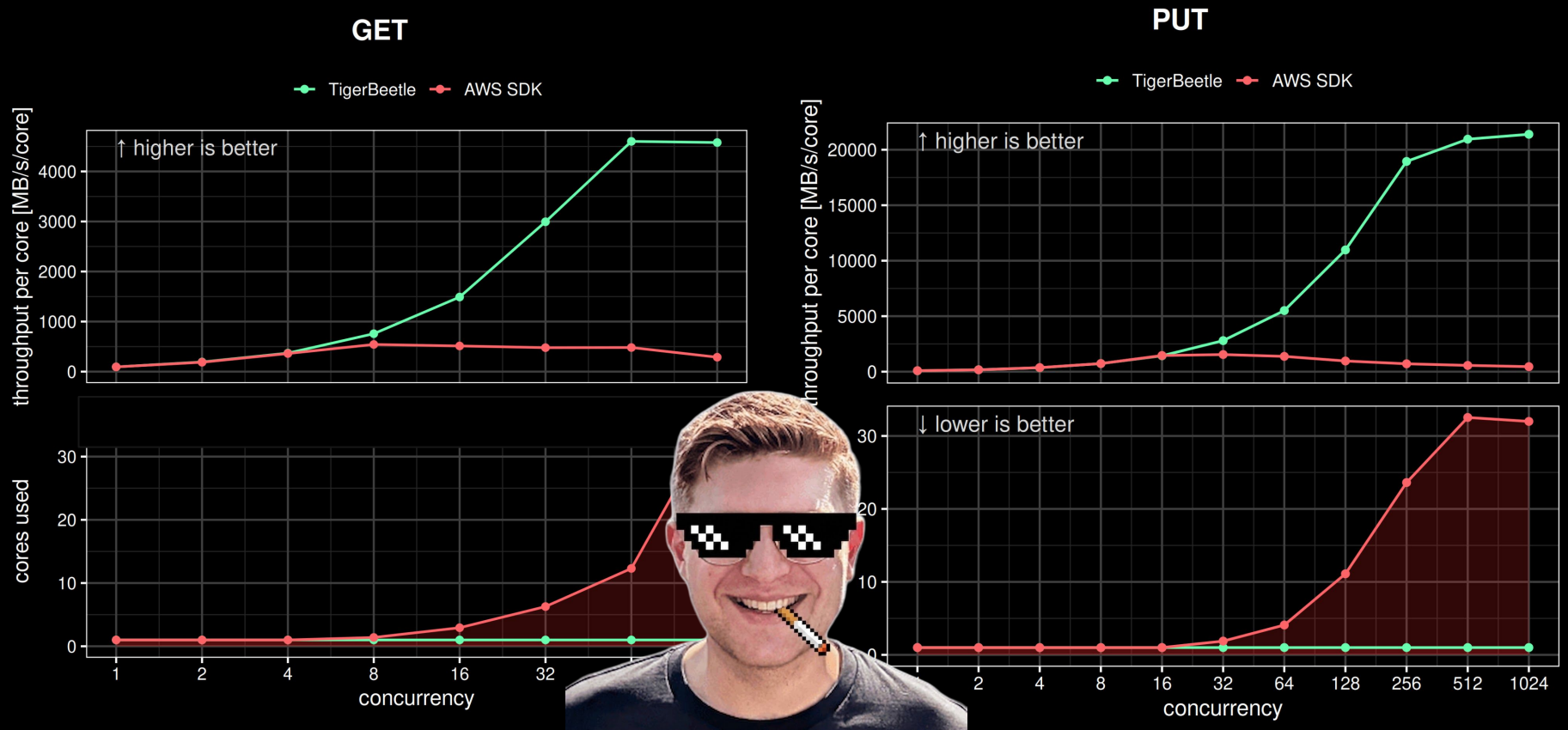
DNS | TLS | HTTP

TCP/IP | Async I/O

Georg's Object Storage Client

- Integrated with our asynchronous I/O
- asynchronous DNS lookups
  - self-paced TTL checks; cache in hashmap
- zero copy send, io_uring registered buffers
  - zero copy recv not yet supported everywhere
- tight integration with encryption library
  - no copying of buffers to prepend metadata etc.
- DST: inject corrupt bits, truncated data, …

What does it get us?



Takeaways

- Question the status quo; start with the interface
- Validate assumptions with napkin math; plot it!
- A good mental model of the hardware is important
- Engineering is as important as complexity theory
- Owning your entire stack is worth it
- If you do it right, it’s investment, not debt



Thanks!

